



Universidad Nacional Autónoma de México

Ingeniería en Computación

Procesamiento digital de señales

Proyecto No. 01

Estudiantes:

Hernández Nativitas Sofia Alejandra - 320107023

Ortiz Briseño Emiliano - 320112179

Ruíz Ramírez Santiago - 320263635

Grupo: 03 Semestre: 2027-I

Ciudad de México, México. Octubre 2026

Índice

1. Introducción	2
2. Planteamiento del problema	2
3. Desarrollo por mariposas	3
3.1. Agrupación de las muestras en cada etapa	4
3.2. Uso de factores de giro	5
3.3. Flujo de datos desde la entrada hasta la salida	6
4. Estructura de las 10 etapas	7
4.1. Organización general de cada etapa	7
4.2. Número de mariposas por etapa	8
4.3. Cambio en las distancias entre índices por nivel	8
5. Ordenamiento de datos	9
5.1. Reordenamiento por inversión de bits (<i>bit-reversal</i>)	9
5.2. Lógica de conversión binaria de 10 bits	11
6. Diagrama o esquema	12
6.1. Esquema ilustrativo a escala reducida (FFT de 8 puntos)	12
6.2. Representación general de la FFT de 1024 puntos	13
7. Complejidad computacional	14
8. Conclusiones	15

1. Introducción

El Análisis de Fourier es una herramienta fundamental en el procesamiento digital de señales (DSP), ya que permite transformar una señal descrita en el dominio del tiempo a su representación equivalente en el dominio de la frecuencia. La Transformada Discreta de Fourier (DFT, por sus siglas en inglés) es la formulación matemática estándar para convertir una secuencia finita de muestras discretas en sus componentes espectrales.

Sin embargo, existe una diferencia fundamental entre la **DFT** y la **Transformada Rápida de Fourier (FFT)**:

- La **DFT** representa la definición matemática explícita de la transformación. Su cálculo directo para un conjunto de N datos requiere evaluar sumatorias complejas punto por punto, lo cual implica un número de operaciones aritméticas (multiplicaciones y adiciones complejas) del orden de $O(N^2)$.
- La **FFT** no es una transformada matemáticamente distinta, sino un conjunto de algoritmos eficientes —como el algoritmo de Cooley-Tukey— concebidos para computar exactamente el mismo resultado que la DFT pero optimizando el uso de recursos computacionales.

La reducción drástica de la complejidad computacional que logra la FFT se justifica por la explotación de las propiedades de simetría y periodicidad del factor de fase o “factor de giro” (*twiddle factor*), $W_N^k = e^{-j\frac{2\pi}{N}k}$. Al aplicar una estrategia de “divide y vencerás” (como el desacoplamiento en radice-2), la secuencia original de tamaño N se descompone recursivamente en transformadas más pequeñas (subsecuencias pares e impares). Esto evita la reevaluación repetitiva de operaciones idénticas y reestructura la complejidad algorítmica de $O(N^2)$ a $O(N \log_2 N)$. Para una señal de longitud considerable, esta diferencia representa un ahorro computacional masivo, haciendo viable el procesamiento de señales en tiempo real en sistemas modernos.

2. Planteamiento del problema

La Transformada Discreta de Fourier (DFT) permite llevar una secuencia discreta $x[n]$ de N muestras del dominio del tiempo al dominio de la frecuencia mediante la suma:

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot W_N^{kn}, \quad \text{para } k = 0, 1, \dots, N-1 \quad (1)$$

donde $W_N = e^{-j\frac{2\pi}{N}}$ es el factor de giro o *twiddle factor*.

Si intentamos calcular esta transformación de manera directa para una señal de $N = 1024$ puntos, el costo computacional se vuelve un problema evidente. Para obtener cada uno de los 1024 componentes de frecuencia $X[k]$, es necesario realizar 1024 multiplicaciones y adiciones complejas. Esto significa que procesar el bloque completo exige $N^2 = 1024^2 = 1,048,576$ operaciones complejas, lo cual resulta ineficiente e impractical para aplicaciones en tiempo real.

Para resolver este cuello de botella, aprovechamos que el tamaño de la muestra es una potencia exacta de dos, ya que $1024 = 2^{10}$. Esta propiedad permite aplicar el algoritmo FFT de Radix-2 mediante la técnica de diezmado en el tiempo. La idea central consiste en descomponer de forma recursiva la secuencia original de 1024 puntos en subsecuencias de muestras pares e impares, reduciendo el problema a transformadas elementales de 2 puntos.

Dado que el exponente es $M = 10$, el proceso completo se organiza en exactamente 10 etapas consecutivas dentro de una red de flujo de datos (*butterfly network*). En cada una de estas 10 etapas se realizan $N/2 = 512$ operaciones de mariposa. Como resultado, el total de multiplicaciones complejas pasa de más de un millón a tan solo:

$$\frac{N}{2} \log_2(N) = 512 \times 10 = 5,120 \text{ operaciones} \quad (2)$$

Esto representa una reducción del trabajo computacional superior al 99.5 %. El objetivo de este reporte es desarrollar e implementar la estructura paso a paso de este algoritmo de 10 etapas para procesar eficientemente la señal de 1024 puntos.

3. Desarrollo por mariposas

Ahora explicaremos a detalle cómo funciona el desarrollo por mariposas. Como mencionamos antes, la idea principal es resolver el problema dividiéndolo en partes pequeñas; la que se encarga de combinar únicamente parejas de 2 datos es la operación de mariposa.

Esta recibe el nombre debido al diagrama que se forma y cómo se conectan los datos cruzándose, simulando la forma de una mariposa. Su funcionamiento se basa en que recibe, por ejemplo, dos datos de entrada de una señal, A y B (A en la parte superior y B en la parte inferior), como se puede ver en la Figura 1.

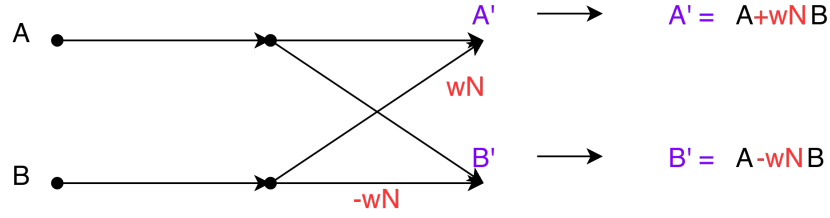


Figura 1: Mariposa de dos valores

El cálculo que se hace en cada mariposa consta de solo tres pasos:

1. **Multiplicación en la rama B:** El dato de entrada de B se multiplica por el factor de giro (W_N). El dato de entrada A pasa sin multiplicarse por nada.
2. **Cálculo de la salida A' :** Para obtener el resultado de A' , se debe sumar el valor original de A más la multiplicación de B por el factor de giro:

$$A' = A + W_N \cdot B \quad (3)$$

3. **Cálculo de la salida B' :** Para obtener el resultado de B' , se toma el valor original de A y en este caso se resta el mismo producto de B por el factor de giro:

$$B' = A - W_N \cdot B \quad (4)$$

La ventaja de usar la mariposa es que te ahorra trabajo, en específico la obtención y multiplicación del valor complejo W_N , ya que se calcula solo una vez y de ahí solo se le agrega un signo negativo cuando se usa de nuevo en la parte inferior.

3.1. Agrupación de las muestras en cada etapa

Como mencionamos arriba, la señal se va dividiendo a la mitad una y otra vez hasta la mínima expresión que es de dos muestras (separando las muestras pares de las impares). Una vez que se tiene todo dividido, las mariposas hacen lo opuesto, es decir van agrupando y juntando las muestras etapa por etapa para ir formando grupos del doble de tamaño.

Para nuestro caso de 1024 muestras tendríamos:

- **Etapa 1:** Las muestras se agrupan de 2 en 2, como tenemos 1024 muestras en total, se forman 512 parejas independientes, cada pareja pasa por una mariposa para calcular una transformada de 2 puntos.

- **Etapa 2:** Los resultados de la Etapa 1 ahora se agrupan de 4 en 4, es decir, se toman dos parejas vecinas de la etapa anterior y se entrelazan usando 2 mariposas para formar bloques de 4 puntos.
- **Etapa 3:** Se juntan dos bloques de 4 puntos para formar grupos de 8 puntos, usando 4 mariposas por cada grupo.
- **Etapas 4 a 9:** Esta misma regla se repite en cada una de las siguientes etapas. Siendo m el número de la etapa, las muestras se organizan en bloques de tamaño 2^m y cada bloque utiliza la mitad de ese número (2^{m-1}) en mariposas para combinar los datos de la etapa previa.
- **Al final (Etapa 10):** Los últimos dos grandes bloques de 512 muestras se unen mediante 512 mariposas para entregar el bloque único de 1024 puntos.

3.2. Uso de factores de giro

En todas las ecuaciones de las mariposas aparece el término W_N , conocido como factor de giro, cuya definición matemática es:

$$W_N = e^{-j\frac{2\pi k}{N}} \quad (5)$$

El factor de giro nos sirve para poder realizar la suma de manera correcta, ya que cuando dividimos la señal original en muestras pares e impares, las muestras impares están desfasadas en el tiempo respecto a las pares. Si solo se sumaran sin hacerles nada se mezclarían datos desalineados.

El factor de giro W_N ajusta el ángulo, multiplicando al dato de la rama B para rotar su fase en el plano complejo y alinearlo antes de sumarlo y/o restarlo con el dato A . La N en el factor de giro es el tamaño total de la transformada, por ejemplo $N = 1024$ en el problema solicitado.

Además, este factor tiene tres propiedades que hacen que la FFT sea rápida:

1. **Caso sin giro ($k = 0$):** Cuando $k = 0$, el factor vale $W_N^0 = e^0 = 1$. Esto ocurre en todas las mariposas de la Etapa 1 y en la primera mariposa de cada grupo en las demás etapas, ahorrándonos esa multiplicación.
2. **Simetría de media vuelta:** Sumarle $N/2$ al exponente k equivale a girar medio círculo (180°), lo que simplemente cambia el signo del resultado: $W_N^{k+N/2} = -W_N^k$. Es por esto que en cada mariposa tan solo se agrega el '−1' en la parte baja de la mariposa, evitando calcular otra exponencial.

3. **Valores repetidos:** Al ser funciones periódicas, los mismos factores de giro se repiten en distintas etapas y grupos, por lo que pueden guardarse en una tabla en memoria sin tener que recalcularlos cada vez.

3.3. Flujo de datos desde la entrada hasta la salida

Para entender el flujo que se toma para realizar una mariposa completa, vamos a seguir el siguiente diagrama.

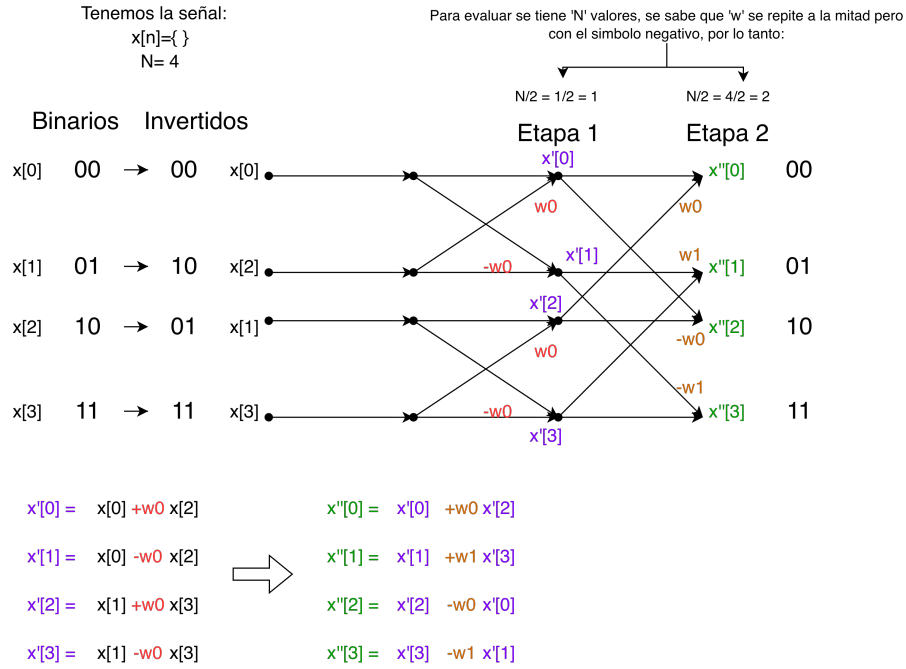


Figura 2: Mariposa de 4 valores

Para empezar, primero vamos a ver tenemos, en este caso nos dan una señal $X[n]$, donde $N = 4$, con esto en mente, sabemos que tendremos que realizar una mariposa de dos etapas, así como que tendremos valores de $X[0]$, $X[1]$, $X[2]$, $X[3]$.

Una vez con esto puesto los datos habrá que organizarlos ya que no entran en orden consecutivo a la mariposa, estos deben de acomodarse primero usando un ordenamiento de inversión de bits (bit-reversal), para ello asignaremos un binario a cada número:

$$X[0] = 00 \quad (6)$$

$$X[1] = 01 \quad (7)$$

$$X[2] = 10 \quad (8)$$

$$X[3] = 11 \quad (9)$$

Ahora les aplicaremos el bit-reversal, quedando de la siguiente manera:

$$00 = X[0] \quad (10)$$

$$10 = X[2] \quad (11)$$

$$01 = X[1] \quad (12)$$

$$11 = X[3] \quad (13)$$

Este será el orden el que entraran los valores en la mariposa, ya de ahí se realizan las operaciones de la etapa 1, marcadas en color morado, que son justo como el ejemplo que estaba arriba, obteniendo los valores de $X'[0]$, $X'[1]$, $X'[2]$, $X'[3]$, como se puede ver en la imagen viene desarrollado la manera de calcular estos mismos, que en si es la suma/resta del valor superior con el valor inferior multiplicado por el factor de giro. Con los valores de la etapa 1, vamos a calcular la etapa 2, estos en color verde del diagrama, obteniendo los valores de la misma forma que los anteriores, solo que ahora se agrupan en grupos de 2, así obteniendo $X''[0]$, $X''[1]$, $X''[2]$, $X''[3]$, así obteniendo los datos ya combinados.

4. Estructura de las 10 etapas

Para poder ver cuantas veces podemos dividir el total de muestras entre 2 para llegar a las parejas de datos y poder realizar las mariposas se calcula mediante el logaritmo base dos del número de muestras:

$$M = \log_2(N) \quad (14)$$

Dado que la señal pedida tiene $N = 1024$ puntos esto nos da en $M = \log_2(1024) = 10$ etapas consecutivas. Cada una de estas etapas como vimos en el ejemplo anterior toma los resultados de la columna anterior y duplica el tamaño de las transformadas parciales hasta reconstruir el bloque completo.

4.1. Organización general de cada etapa

En cada etapa la columna de 1024 datos se divide en subgrupos o bloques, para cada etapa m , cada grupo resuelve una DFT de tamaño $L_m = 2^m$, por ejemplo en la etapa 1 los bloques son de son de $2^1 = 2$ muestras, en la etapa 2 son de $2^2 = 4$ y así sucesivamente hasta

la etapa 10 de un solo bloque de $2^{10} = 1024$ muestras, así mismo para saber la cantidad de grupos que tenemos en cada etapa, tenemos:

$$G_m = \frac{N}{2^m} = \frac{1024}{2^m} = 2^{10-m} \quad (15)$$

Esto muestra que en la etapa 1 tenemos 512 grupos pequeños, de 2 muestras cada uno, en cada nueva etapa los vecinos se unen de dos en dos, por lo que la cantidad se reduce a la mitad en cada paso, de 512, a 256, a 128, a 64, etc, hasta llegar a 1 en la etapa 10

4.2. Número de mariposas por etapa

Como se menciono antes cada operación de mariposa en cada nivel necesita 2 datos de entrada y dos de salida, en este caso con las 1024 muestras el número total de mariposas por etapa es siempre constante, es decir siempre habra 512 mariposas esto lo sabemos debido a:

$$B_{\text{total}} = \frac{N}{2} = \frac{1024}{2} = 512 \text{ mariposas en cada etapa} \quad (16)$$

Esto debido a que en todas las etapas participan las mismas 1024 muestras y ninguna muestra puede quedarse sin usar, pero lo que si cambia es en cada nivel como se reparten las 512 mariposas, esto lo podemos saber con la formula:

$$B_{\text{grupo}} = 2^{m-1} \text{ mariposas por grupo} \quad (17)$$

Con esto sabemos que en la etapa 1 hay 512 mariposas y cada grupo tiene 1 sola mariposa, en la etapa 2 hay 256 grupos con 2 mariposas cada uno, en el grupo 3 hay 128 grupos con 4 mariposas cada uno y así sucesivamente hasta la etapa 10 que solo hay 1 grupo con 512 mariposas.

4.3. Cambio en las distancias entre índices por nivel

Otra cosa que va a ir cambiando conforma avanza la mariposa es la distancia entre los índices(d_m) que conectan los valores para la crear la nueva mariposa, este valor cambia de la misma forma que la cantidad de mariposas por grupo, es decir:

$$d_m = 2^{m-1} \quad (18)$$

Es decir, si la rama superior de una mariposa está en la posición (índice) p , la rama inferior con la que se va a cruzar siempre estará en la posición $q = p + 2^{m-1}$: Es decir en la etapa 1, la

distancia entre las mariposas es $d_1 = 2^{1-1} = 2^0 = 1$, es decir la que tiene directamente abajo, para la etapa 2 la distancia se duplica a $d_2 = 2^{2-1} = 2$, es decir ahora se saltan un renglon para cruzarse, en la etapa 3 duplicara de nuevo la distancia ahora saltandose 4 renglones y así sucesivamente hasta llegar a la etapa 10, en la cual se llegara a la separación máxima de 512.

Así mismo tambien cambian los factores de giro en cada etapa este va cambiando en cada etapa, ya que cada grupo necesita 2^{m-1} factores de giro diferentes, es decir uno para cada mariposa del grupo. Para el exponente k este empieza siempre en 0 y va dando vueltas de tamaño:

$$\Delta k = \frac{1024}{2^m} = 2^{10-m} \quad (19)$$

En la etapa 1 el salto es de 512, pero como solo hay 1 mariposa todas usan $W = 1$, para el caso de la etapa 2 el salto es de 256, por lo que las 2 mariposas de cada etapa usan W_0 y W_{256} , en la etapa 3 el salto es de 128, por lo que se usan 4 valores, W_0 , W_{128} , W_{256} y W_{384} , así sucesivamente hasta la etapa 10 que el salto es de 1, así que se usará desde el W_0 hasta el W_{511} .

A continuación se muestra el resumen de las 10 etapas con sus parametros de tamaño del bloque, número de grupos, mariposas por grupo, mariposas totales, la distancia entre sus indices y sus factores de giro de cada una.

Tabla 1: Resumen de las 10 etapas para de $N = 1024$ puntos.

Etapa (m)	Tamaño de bloque ($L_m = 2^m$)	Núm. de grupos ($1024/2^m$)	Mariposas por grupo (2^{m-1})	Total de mariposas ($N/2$)	Distancia entre índices (d_m) (2^{m-1})	Salto del exponente ($\Delta k = 2^{10-m}$)	Factores de giro W_k usados en cada grupo ($k = 0, \Delta k, 2\Delta k, \dots$)
1	2	512	1	512	1	512	W_0
2	4	256	2	512	2	256	W_0, W_{256}
3	8	128	4	512	4	128	$W_0, W_{128}, W_{256}, W_{384}$
4	16	64	8	512	8	64	$W_0, W_{64}, W_{128}, \dots, W_{448}$
5	32	32	16	512	16	32	$W_0, W_{32}, W_{64}, \dots, W_{480}$
6	64	16	32	512	32	16	$W_0, W_{16}, W_{32}, \dots, W_{496}$
7	128	8	64	512	64	8	$W_0, W_8, W_{16}, \dots, W_{504}$
8	256	4	128	512	128	4	$W_0, W_4, W_8, \dots, W_{508}$
9	512	2	256	512	256	2	$W_0, W_2, W_4, \dots, W_{510}$
10	1024	1	512	512	512	1	$W_0, W_1, W_2, \dots, W_{511}$

5. Ordenamiento de datos

5.1. Reordenamiento por inversión de bits (*bit-reversal*)

En el esquema radix-2 con decimación en el tiempo que se desarrolló, la secuencia de entrada se separa una y otra vez en muestras de índice par e impar hasta llegar a parejas.

Esa separación deja las muestras en un orden distinto al natural, y es justamente en ese orden en el que deben entrar a la primera etapa de mariposas. Con este acomodo cada mariposa guarda sus dos resultados en las mismas posiciones donde leyó sus dos entradas, por lo que todo el cálculo se realiza sobre un solo arreglo (*in-place*) y la salida $X[k]$ queda en orden natural.

El orden requerido surge de la siguiente lógica. En la primera separación, el bit menos significativo del índice decide en qué mitad cae cada muestra: si vale 0 (índice par) va a la mitad superior y si vale 1 (índice impar) va a la mitad inferior. En la siguiente separación, el segundo bit decide la mitad dentro de cada mitad, y así sucesivamente. Por lo tanto, el primer bit que se lee (el menos significativo) termina siendo el más significativo de la nueva posición, y el último bit leído termina siendo el menos significativo; es decir, los bits del índice se leen en orden inverso.

Por ejemplo, para $x[6]$ con $N = 8$ se tiene $6 = 110_2$: el bit menos significativo es 0 (mitad superior), el siguiente es 1 (mitad inferior de esa mitad) y el último es 1 (mitad inferior de nuevo), de modo que su nueva posición es $011_2 = 3$. La Tabla 2 muestra el reordenamiento completo para $N = 8$.

Índice n	Binario (3 bits)	Bits invertidos	Nueva posición
0	000	000	0
1	001	100	4
2	010	010	2
3	011	110	6
4	100	001	1
5	101	101	5
6	110	011	3
7	111	111	7

Tabla 2: Inversión de bits para $N = 8$. La nueva posición de $x[n]$ es el valor del número binario invertido.

Con esto, la secuencia entra a la primera etapa en el orden $x[0], x[4], x[2], x[6], x[1], x[5], x[3], x[7]$, que es el que se utiliza en el diagrama de la sección siguiente. De forma análoga, en el ejemplo de $N = 4$ el orden resultante es $x[0], x[2], x[1], x[3]$.

Esta permutación es su propia inversa: si se aplica dos veces se recupera el orden original. Por ello, en una implementación basta con intercambiar cada pareja de posiciones $(n, \text{rev}(n))$ una sola vez, y los índices cuya representación binaria es simétrica (palíndromos) permanecen en su lugar. Para $N = 1024$ hay $2^5 = 32$ índices que no se mueven y $(1024 - 32)/2 = 496$ intercambios.

Finalmente, cabe señalar que el reordenamiento depende de la variante del algoritmo. Como en este trabajo se utiliza decimación en el tiempo, la inversión de bits se aplica a la entrada y la salida sale en orden natural. En la variante de decimación en frecuencia ocurre lo contrario: la entrada entra en orden natural y el reordenamiento aparece en la salida.

5.2. Lógica de conversión binaria de 10 bits

Dado que $N = 1024 = 2^{10}$, cada índice $n \in \{0, 1, \dots, 1023\}$ se representa exactamente con 10 bits, incluyendo los ceros a la izquierda:

$$n = \sum_{i=0}^9 b_i 2^i = (b_9 b_8 \dots b_1 b_0)_2 \quad (20)$$

La inversión de bits intercambia las posiciones de los coeficientes b_i , de modo que el bit b_0 pasa a ser el más significativo y el bit b_9 el menos significativo:

$$\text{rev}_{10}(n) = \sum_{i=0}^9 b_i 2^{9-i} = (b_0 b_1 \dots b_8 b_9)_2 \quad (21)$$

El procedimiento para obtener la nueva posición de una muestra es:

1. Escribir el índice n en binario con 10 bits, completando con ceros a la izquierda.
2. Leer los bits de derecha a izquierda y escribirlos en ese orden, de izquierda a derecha.
3. Convertir el número binario obtenido a decimal; ese valor es la posición $\text{rev}_{10}(n)$.

Es importante fijar siempre la longitud en 10 bits, pues los ceros a la izquierda también se invierten y modifican el resultado. Por ejemplo, el índice $n = 1$ se escribe como 0000000001_2 y al invertirlo resulta $1000000000_2 = 512$; si se usaran solo 3 bits se obtendría 4, que corresponde al caso de $N = 8$. La Tabla 3 muestra algunos ejemplos.

Índice n	Binario (10 bits)	Bits invertidos	$\text{rev}_{10}(n)$
0	0000000000	0000000000	0
1	0000000001	1000000000	512
2	0000000010	0100000000	256
3	0000000011	1100000000	768
5	0000000101	1010000000	640
1000	1111101000	0001011111	95
1023	1111111111	1111111111	1023

Tabla 3: Ejemplos de inversión de bits para $N = 1024$.

De esta tabla se desprende una consecuencia directa: la primera mariposa de la etapa 1 combina las muestras $x[0]$ y $x[512]$ (que ocupan las posiciones 0 y 1), y la segunda combina $x[256]$ y $x[768]$ (posiciones 2 y 3). Es decir, las muestras que se emparejan primero están separadas $N/2$ en el tiempo, como corresponde a la decimación en el tiempo.

En software, la inversión se calcula repitiendo 10 veces los siguientes pasos: extraer el bit menos significativo de n con una operación AND con 1, desplazar el resultado un lugar a la izquierda para agregar ese bit a su derecha, y desplazar n un lugar a la derecha para pasar al siguiente bit. Así se evita calcular el valor con potencias de 2 y solo se usan operaciones sobre bits.

6. Diagrama o esquema

Para mostrar la organización del algoritmo se presentan dos esquemas: primero un diagrama completo a escala reducida ($N = 8$), en el que se pueden seguir todas las conexiones, y después una representación general de la estructura de 1024 puntos, que utiliza exactamente las mismas reglas.

6.1. Esquema ilustrativo a escala reducida (FFT de 8 puntos)

Para $N = 8$ se tiene $M = \log_2(8) = 3$ etapas con $N/2 = 4$ mariposas cada una. La Figura 3 muestra el flujo completo: las muestras entran en orden *bit-reversal* por la izquierda, atraviesan las tres etapas (cada color corresponde a una etapa) y salen en orden natural por la derecha. En cada mariposa, la rama inferior se multiplica por el factor de giro indicado antes de sumarse y restarse con la rama superior.

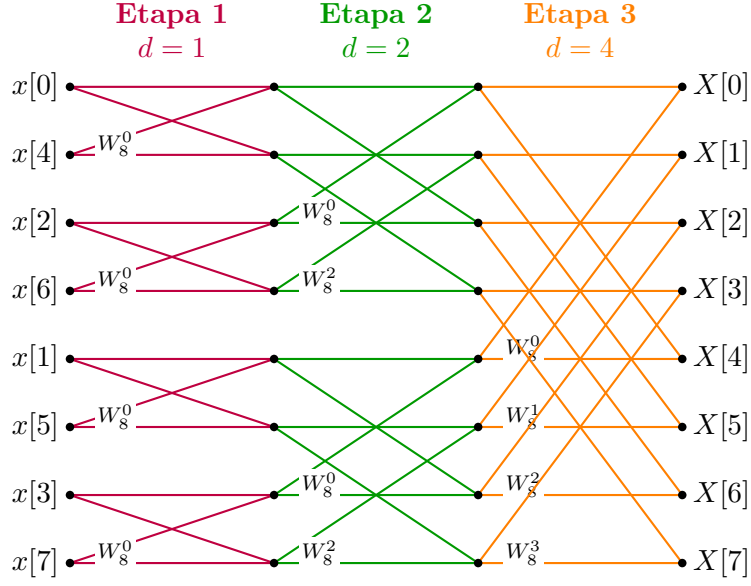


Figura 3: Diagrama de mariposas de una FFT de 8 puntos (radix-2, decimación en el tiempo). d es la distancia entre los índices de cada mariposa y W_8^e el factor de giro que multiplica a la rama inferior.

A partir de la Figura 3 se observa que:

- En la **etapa 1** las mariposas combinan posiciones vecinas ($d = 1$) y todas usan $W_8^0 = 1$.
- En la **etapa 2** la distancia se duplica ($d = 2$) y se utilizan W_8^0 y W_8^2 , es decir, un factor distinto por cada mariposa de un mismo grupo.
- En la **etapa 3** se forma el único grupo de 8 puntos ($d = 4$) y se usan W_8^0 , W_8^1 , W_8^2 y W_8^3 .

Estas observaciones coinciden con las reglas generales de la sección anterior: en la etapa m la distancia es $d_m = 2^{m-1}$ y los exponentes avanzan de $\Delta k = N/2^m$ en Δk (para $N = 8$: saltos de 4, 2 y 1). Estas mismas reglas se aplican sin cambios para $N = 1024$.

6.2. Representación general de la FFT de 1024 puntos

Dibujar las 1024 líneas de cada etapa no aporta claridad, por lo que la estructura se representa de dos maneras. La Figura 4 muestra el flujo de bloques completo: reordenamiento por inversión de bits seguido de las 10 etapas de mariposas, cada una con 512 mariposas y su propia distancia $d_m = 2^{m-1}$.

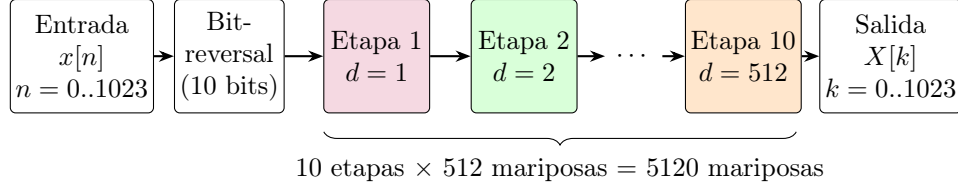


Figura 4: Estructura general de la FFT de 1024 puntos: inversión de bits seguida de 10 etapas de mariposas.

Para ver cómo cambian las conexiones entre etapas, la Figura 5 muestra algunas mariposas de la primera y de la última etapa. Los números son posiciones del arreglo (no tiempos): en la etapa 1 las mariposas unen posiciones contiguas, y todas usan $W_{1024}^0 = 1$; en la etapa 10 unen la posición p con la $p + 512$ y usan los factores $W_{1024}^0, W_{1024}^1, \dots, W_{1024}^{511}$.

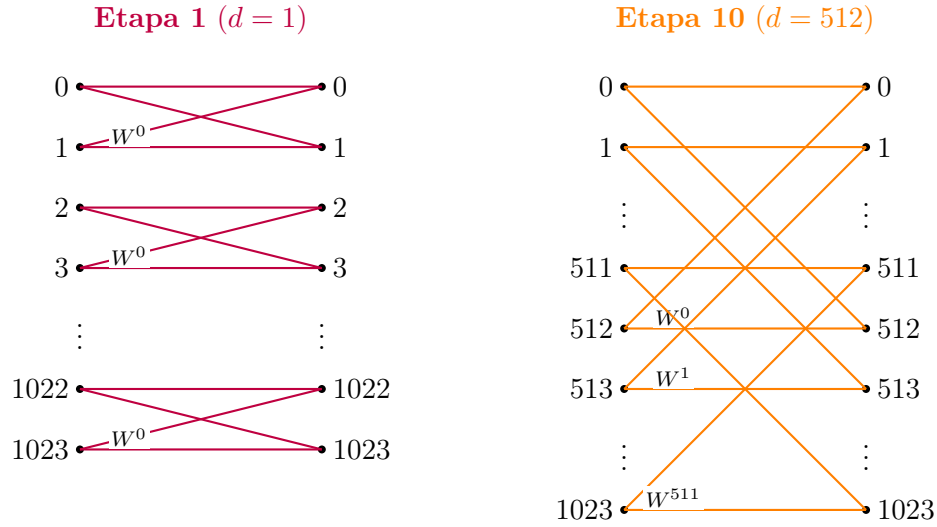


Figura 5: Detalle de la conexión de las mariposas en la primera y en la última etapa de la FFT de 1024 puntos (se muestran solo algunas mariposas). W^e denota W_{1024}^e .

En resumen, el flujo de datos de la FFT de 1024 puntos es siempre el mismo: las muestras se reordenan por inversión de bits, y en cada una de las 10 etapas se aplican 512 mariposas idénticas en su forma, que solo cambian en la distancia $d_m = 2^{m-1}$ entre sus índices y en los factores de giro W_{1024}^k que utilizan.

7. Complejidad computacional

El cálculo directo de la Transformada Discreta de Fourier (DFT) para una señal de $N = 1024$ puntos requiere evaluar N sumatorias de N términos cada una, lo que implica una complejidad computacional de orden $O(N^2)$. Esto equivale exactamente a:

$$N^2 = 1024^2 = 1,048,576 \quad \text{multiplicaciones y adiciones complejas} \quad (22)$$

En contraste, la implementación de la FFT mediante el algoritmo de Radix-2 aprovecha la simetría y periodicidad de los factores de giro, reduciendo la complejidad a $O(N \log_2 N)$. Para $N = 1024$ (10 etapas con 512 mariposas por etapa), el total de operaciones complejas disminuye a:

$$\frac{N}{2} \log_2(N) = \frac{1024}{2} \times 10 = 5,120 \quad \text{operaciones de mariposa} \quad (23)$$

Esta optimización representa una reducción computacional del 99.51 % respecto al cálculo directo de la DFT. En términos analíticos, mientras que la DFT exige un número de operaciones que crece de forma cuadrática con el tamaño de la muestra, la FFT mantiene un crecimiento casi lineal, haciendo factible el procesamiento de señales de gran volumen sin saturar la memoria ni los recursos de procesamiento.

8. Conclusiones

El desarrollo de este reporte permitió analizar cuantitativamente la eficiencia del algoritmo FFT de 1024 puntos mediante operaciones tipo mariposa. Se demostró que la descomposición recursiva en 10 etapas en base 2 transforma un problema de alto costo computacional en una estructura altamente optimizada y simétrica. Esta reducción drástica de operaciones es el pilar que posibilita el procesamiento digital de señales en tiempo real dentro de sistemas modernos de telecomunicaciones, procesamiento de audio e imágenes.

Asimismo, la estructura modular y regular de la red de mariposas resulta ideal para su implementación práctica. En software, facilita el aprovechamiento del paralelismo a nivel de instrucciones y el acceso eficiente a la memoria caché. En hardware (como FPGAs o DSPs dedicados), la naturaleza repetitiva de las etapas permite diseñar arquitecturas canalizadas (pipelined) y de procesamiento en paralelo, reduciendo la latencia al mínimo y maximizando el rendimiento global del sistema.

Referencias

- [1] Weisstein, E. W. (s. f.). *Fast Fourier Transform*. MathWorld—A Wolfram Web Resource. <https://mathworld.wolfram.com/FastFourierTransform.html>

- [2] Smith, S. W. (s. f.). *Chapter 12: The Fast Fourier Transform – How the FFT works*. The Scientist and Engineer’s Guide to Digital Signal Processing. <https://www.dspguide.com/ch12/2.htm>
- [3] Massachusetts Institute of Technology [MIT OpenCourseWare]. (2008). *The Radix-2 FFT with Decimation in Time (2.161 Signal Processing: Continuous and Discrete)*. https://ocw.mit.edu/courses/2-161-signal-processing-continuous-and-discrete-fall-2008/2b3897bb64743815afae1543066ed4ff_fft.pdf
- [4] Newman, M. (2024). *How the Cooley-Tukey FFT Algorithm Works – Part 3: The Inner Butterfly*. DSPRelated. <https://www.dsprelated.com/showarticle/1712.php>
- [5] Arzate Gordillo, J. [Jacqueline Arzate Gordillo]. (20 de enero de 2021). *Algoritmo FFT para una secuencia de 4 puntos* [Video]. YouTube. <https://www.youtube.com/watch?v=tgUroiIIW9A>